Submission Worksheet

IT202-450-M2026 / module-04-worksheet-sql-todo-challenge

Submission Data

Course	IT202-450-M2026
Assignment	Module 04 Worksheet: SQL Todo Challenge
Student	Shakir A. (sa3327)
Status	submitted
Worksheet Progress	98%
Potential Grade	NaN/NaN (NaN%)
Started	[object Object]
Updated	7/1/2026, 1:35:03 PM

Grading Link
https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-04-worksheet-sql-todo-challenge/grading/sa3327

View Link
https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-04-worksheet-sql-todo-challenge/sa3327

Instructions

Complete the Module 4 SQL todo challenge in the homework branch, then collect focused evidence for the database setup, create page, pending page, and completed page. It's best to solve and test the PHP files first, then fill out the worksheet; however, you may also collect evidence as each part is solved.

Assignment Setup

- Starter reference: Module 4 Homework branch
- Required branch: `Module04-Homework`
- Required starting branch: `qa`
- Required homework folder: `public_html/m04/hw/`
- Required SQL setup folder: root `sql/`
- Starter files to copy:
  - Root `sql/000_create_table_todo.sql`
  - `todos/create.php`
  - `todos/pending.php`
  - `todos/completed.php`
  - `nav.php`
  - The two provided `index.php` files for the `m04/hw` and `todos` folders
- Recommended order:
  - Read the whole worksheet.
  - Copy the starter files; commit the baseline.
  - Run the existing project database init runner so the root SQL file is applied.
  - Add UCID/date and planning comments where edits are allowed; commit those comments.
  - Solve each page, test through `qa`, then collect the worksheet evidence.

Git Workflow Checklist

1. Check out `qa`

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the homework branch with `git checkout -b Module04-Homework`.
4. Copy the PHP starter files into `public_html/m04/hw/` and the SQL starter file into root `sql/`.
5. Commit the baseline starter files before solving the challenge.
6. Push the branch with `git push origin Module04-Homework`.
7. Open a pull request from `Module04-Homework` into `qa` and keep it open while you work.
8. Run the existing project database init runner (visit `/project/run_init_db.php`) after the SQL file is copied.
    - Or run `php sql/init_db.php` in the command line from the repo root, although the output won't be rendered nicely
9. Add planning comments before the final code for every challenge area.
10. Commit planning comments before the finished solution for each challenge area.
11. Commit working code in small pieces as each page starts working.
12. After all pages pass locally, merge the homework pull request into `qa`.
13. Gather evidence from the deployed `qa` site and fill out the worksheet.
14. Export the worksheet PDF from the Learn Courses Platform.
15. Save the PDF in the repository under `docs/module04/`.
16. Add, commit, and push the PDF to the `Module04-Homework` branch.
    - `git add .`
    - `git commit -m "Add module 4 worksheet PDF"`
    - `git push origin Module04-Homework`
17. Upload the same PDF to Canvas.
18. Create and merge a pull request from `qa` into `prod` so the anticipated production URLs become live.
19. Switch back to `qa` with `git checkout qa`.
20. Pull the latest `qa` branch with `git pull origin qa`.

Shared Requirements

Apply these requirements to each challenge area:

- Only edit code where the starter comments say edits are allowed.
- Add a comment with your UCID, date, and a brief summary of the solution approach.
- Add planning comments before the solution code.
- Commit the planning-comment stage before the final solution.
- Update the visible UCID in `nav.php` where the starter asks for it.
- Validate incoming form data before database writes.
    - Use the todo table shape to guide validation: `task` should be present and fit the column length, `due` should be a valid database date, and the final `assigned` value should be non-empty text that fits `VARCHAR(60)` or fall back to `self`.
    - For update actions, confirm submitted todo identifiers are valid before using them in a query.
- Use PDO named placeholders for inserts and updates.
- Create enough distinct todo records through your pages to demonstrate at least five pending records and at least five completed records in the deployed evidence.
- Test the file through the supported server path before collecting output evidence. For deployed evidence, use the working `qa` URL after the site has deployed.
- Use your own code, output, branch, pull request, database behavior, and deployed URLs for evidence.

Challenge Evidence Format

Most challenge areas use the same evidence pattern:

Challenge Evidence Format

Most challenge areas use the same evidence pattern:

- Images:
  - Show the relevant code and the matching browser output.
  - Make sure the UCID/date comment is visible in the code evidence.
  - Make sure the deployed page URL is visible in the browser output evidence when browser output is requested.
  - One combined screenshot is acceptable if the code and output are both readable.
  - Use two screenshots when one image would be hard to read.
- URLs:
  - Provide the direct GitHub file link from the homework branch.
  - Provide the anticipated production URL for the PHP file.
  - Capture the working `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - Use the tested Module 4 homework path, such as `/m04/hw/todos/create.php`, `/m04/hw/todos/pending.php`, or `/m04/hw/todos/completed.php`.
  - Each file URL must end in `.php`; zero credit for that URL entry if it does not.
- Response:
  - Explain the logic in your own words.
  - Describe the steps or decisions your PHP or SQL makes.
  - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the homework branch has been pushed.
- Confirm the pull request into `qa` exists.
- Confirm the baseline, planning, and solution commits are visible.
- Confirm the database init runner applied the todo SQL file or reported that already-applied statements were blocked or skipped.
- Confirm create, pending, and completed pages were reviewed on `qa` before promotion.
- Confirm the anticipated production URLs use the same path as the tested `qa` URLs with `-qa` changed to `-prod`.
- Confirm the worksheet PDF is committed under `docs/module04/` on the homework branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress 98%

Section #1: (1 pt.) Prep

100%

- File to run: the existing project database init runner, such as `/project/run_init_db.php`.
- Goal: trigger the provided database setup so the `M4_Todos` table exists before the page code is tested.
- Expected behavior: the output should show the todo SQL file running successfully or being blocked/skipped because it already ran.
- Also show that the copied starter files are present in your editor.

Task #1 (0.5 pts.) – Database Init Output

100%

Details

- Show the browser output of the database init runner.
- The output should show `000_create_table_todo.sql` running successfully, blocked as already applied, or skipped if the setup already ran.

- Show the browser output of the database init runner.
- The output should show `000_create_table_todo.sql` running successfully, blocked as already applied, or skipped if the setup already ran.

database helper

Task #2 (0.5 pts.) – Project Files Present

100%

Details

- Show the project view in your editor.
- The screenshot should show the root `sql/000_create_table_todo.sql` file and the `public_html/m04/hw/` starter files are present.

vscode

Section #2: (3 pts.) Create Page

79%

- File to edit and test: `public_html/m04/hw/todos/create.php`.
- Goal: build the todo creation form, validate submitted values, and insert valid todo records.
- Expected form behavior:
  - The HTML form includes fields that match the todo table and starter comments.
  - Field names match the PHP code that reads the submitted values.
  - Input types are chosen based on the kind of data each column expects.
  - The assigned value defaults to `self` when a valid assigned value is not provided.
- Expected PHP behavior:
  - Incoming data is validated against the SQL table definition.
  - `task` is checked as a required text value that fits the table column.
  - `due` is checked as a date value the database can store.
  - `assigned` is checked as non-empty text that fits `VARCHAR(60)`, with `self` used as the safe fallback.
  - Database-managed values such as `id`, completion state, and timestamps are not taken from the form.
  - Invalid data sets `$is_valid` to false.
  - Invalid data produces a user-friendly message explaining what was wrong.
  - Valid data is inserted with a PDO prepared statement and named placeholders.

Task #1 (0.75 pts.) – HTML Form Challenge

100%

Combo #1

Part 1: Images 100%

Details

Part 1: Images 100%

Details

- Show the code snippet for the completed HTML form.
- Include the field names and the default behavior for the assigned input.

form code

Part 2: Response 100%

Details

- Explain the field names you used.
- Explain why the input types fit the expected data.
- Explain how `self` becomes the default assigned value.
- Do not restate the prompt.

Response

The names of the input fields are so that they match the names of the php code from the $_GET array. The text input is used for task and assigned since they store text and a date field is used for due because its a date. The assigned field is value="self" so if the user doesnt change it, it is the default value.

Supports **bold**, *italic*, [links](url), `code`, etc. 313 chars

Task #2 (0.75 pts.) – Form Submission PHP Validations

94%

Combo #1

Part 1: Images 87%

Details

- Show the code that validates incoming data.
- Show the code that sets `$is_valid` to false when needed.
- Show the user-friendly error messages.
- Include browser evidence from the deployed `qa` page demonstrating the task, due date, and assigned-value validations.
- Make sure the browser address bar is visible.

validate code

qa

Part 2: Response 100%

Details

- Explain your validation logic.
- Explain how your code decides when the task, due date, or assigned value is invalid.
- Explain how the assigned field falls back to `self`.
- Do not restate the prompt.

Response

The code checks the input against the SQL table before attempting to insert anything. The task is invalid if its empty, longer than 128 charts, invalid date, assigned value is > 60 chars. If the assigned field is empty, the code will automatically set it to "self" so there salsways a valid value.

Supports **bold**, *italic*, [links](url), `code`, etc. 298 chars

Task #3 (0.75 pts.) – Insert Query

100%

Combo #1

Part 1: Images 100%

Details

- Show the code snippet for the `INSERT` query.
- Show the PDO named placeholders.
- Show the values being bound or passed to the prepared statement.

insert query

Part 2: Response 100%

Details

- Explain how the insert query maps form values to table columns.
- Explain why named placeholders are used.
- Do not restate the prompt.

Response

The INSERT query takes values from the task, due, assigned form fields and puts it into matching colums of M4_Todos. Named placeholders seperate SQL statements from the user input.

Supports **bold**, *italic*, [links](url), `code`, etc. 181 chars

Task #4 (+ 1 pt.) – Create Page Extra Credit: Unique Constraint Errors

Combo #1

Part 1: Images 0%

Details

- Show the code related to handling unique constraint errors.
- Show the user-friendly message for that database error.

No images submitted yet.

Part 2: Response 0%

Details

- Explain how your code detects the unique constraint error.
- Explain how your code turns the database error into a user-friendly message.
- Do not restate the prompt.

Response

Missing Response

Supports **bold**, *italic*, [links](url), `code`, etc. 0 chars

Task #5 (0.75 pts.) – Create Page URLs

100%

Details

- Paste the direct GitHub file link from the homework branch.
  - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
  - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module04-Homework/public_html/m04/hw/todos/create.php

URL #2
https://sa3327-it202-450-m2026-prod.onrender.com/m04/hw/todos/create.php

Section #3: (3 pts.) Pending Page

94%

- File to edit and test: `public_html/m04/hw/todos/pending.php`.
- Goal: list incomplete todo records and allow the intended record to be marked complete.
- Expected fetch behavior:

- File to edit and test: `public_html/m04/hw/todos/pending.php`.
- Goal: list incomplete todo records and allow the intended record to be marked complete.
- Expected fetch behavior:
  - Select columns in the same order as the provided HTML table.
  - Calculate a virtual `days_offset` value from the due date for the status display.
  - Use the sign direction expected by the starter display: future due dates show as due later, and past due dates show as overdue.
  - Do not select anything special for the actions column.
  - Show only todo records that are not completed.
  - Order records by the due dates that are soonest.
  - Demonstrate at least five records in the deployed `qa` output.
- Expected update behavior:
  - Validate the submitted todo `id` before attempting the update.
  - Update only the selected todo item.
  - Use the todo `id` with a PDO named placeholder.
  - Mark the item complete.
  - Set the completed date field to today's date.
  - Include a condition so already completed records are not updated again.

Task #1 (1 pt.) – Fetch Pending Todos

94%

Combo #1

Part 1: Images 87%

Details

- Show the SQL or PHP code that fetches pending todos.
- Show the virtual `days_offset` expression.
- Show browser output from the deployed `qa` page with five different pending records.
- Make sure the browser address bar is visible.

```
$query =
"SELECT id, task, due,
DATEDIFF(due, CURRENT_DATE) AS days_offset,
assigned
FROM M4_Todos
WHERE is_complete = 0
ORDER BY due ASC"; // edit this
$params = []; // apply mapping
```

query

qa

Part 2: Response 100%

Details

- Explain how the query selects the table columns.
- Explain how the query filters incomplete records.
- Explain how `days_offset` is calculated and how its sign matches the status display.
- Explain how the ordering works.
- Do not restate the prompt.

Response

The query selects id, task, due, days_offset, and assigned in the same order as the HTML table. WHERE is_completed = 0 limits the results to todos that have not been marked as complete. days_offset is calculated with DATEDIFF(due, CURRENT_DATE) which makes future dates positive values and past due ones negative which matches the status messages of the PHP code. ORDER BY due ASC sorts the recordys by due date.

Supports **bold**, *italic*, [links](url), `code`, etc. 413 chars

Supports **bold**, *italic*, [links](url), `code`, etc. 413 chars

Task #2 (1 pt.) – Complete Todo Update

88%

Combo #1

Part 1: Images 75%

Details

- Show the SQL or PHP code that marks a todo complete.
- Show the named placeholder for the todo `id`.
- Show the validation or guard that keeps invalid todo ids from being used.
- Show the condition that prevents already completed records from being updated again.

```
if (!is_numeric($id)) {
    echo "Invalid todo id.";
    return;
}

$query =
    "UPDATE MA_Todos
    SET is_complete = 1,
    completed = CURRENT_DATE
    WHERE id = :id
    AND is_complete = 0";

$params = [
    ':id' => $id
];
```

Missing caption

Part 2: Response 100%

Details

- Explain how the update targets only one todo.
- Explain how the submitted todo id is checked before the update.
- Explain how the completed date is set.
- Explain why the extra incomplete-record condition is included.
- Do not restate the prompt.

Response

WHERE id = :id make sure that the only the todo with the submitted ID is updated. THe code checks is_numeric($id) before runnying the query so only a numberic todo ID is used. THe completed column is set to CURRENT_DATE, which stores the date when the todo is completed. AND is_complete = 0 prevents todos that are already marked as complete from being updated.

Supports **bold**, *italic*, [links](url), `code`, etc. 361 chars

Task #3 (1 pt.) – Pending Page URLs

100%

Details

- Paste the direct GitHub file link from the homework branch.
  - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
  - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - The production URL must end in `.php`; zero credit for this URL if it does not.

- Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
- The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module04-Homework/public_html/m04/hw/todos/pending.php 👍

URL #2
https://sa3327-it202-450-m2026-prod.onrender.com/m04/hw/todos/pending.php 👍

Section #4: (2 pts.) Completed Page

100%

- File to edit and test: `public_html/m04/hw/todos/completed.php`.
- Goal: list completed todo records with the expected table columns and status calculations.
- Expected fetch behavior:
  - Select columns in the same order as the provided HTML table.
  - Extract only the date portion from the completed column.
  - Calculate a virtual `days_offset` value from the completed date for the status display.
  - Use the sign direction expected by the starter display: recent completed dates show as completed day counts, and overdue cases still display as overdue.
  - Show only todo records that are completed.
  - Order records by most recently completed and most recently due.
  - Demonstrate at least five records in the deployed `qa` output.

Task #1 (1 pt.) – Fetch Completed Todos

100%

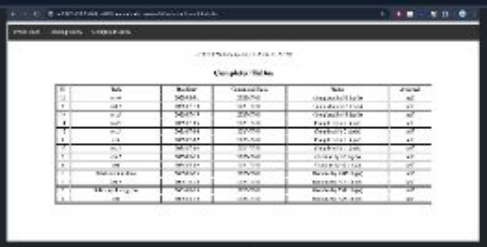
Combo #1

Part 1: Images 100%

Details

- Show the SQL or PHP code that fetches completed todos.
- Show the expression that extracts the date portion from the completed column.
- Show the virtual `days_offset` expression.
- Show browser output from the deployed `qa` page with five different completed records.
- Make sure the browser address bar is visible.

code



qa site

Part 2: Response 100%

Details

- Explain how the query selects the table columns.
- Explain how the query filters completed records.
- Explain how the completed date and `days_offset` values are calculated, including how the sign matches the status display.
- Explain how the ordering works.
- Do not restate the prompt.

- Explain how the ordering works.
- Do not restate the prompt.

Response

The query selects id, task, due, completed, days_offset and assigned columns in the same order as the HTML table. WHERE is_complete=1 filters the results so that only complete todo records are shown. The DATE(completed) pulls the date portion from the completed timestamp, and DATEDIFF(due, DATE(completed)) measures the days from the due date to the completed date. If this number is positive or 0 then it is completed on or before the due date, if negative, it was completed late and will match the status message above. ORDER BY completed DESC, due DESC sorts the records by the most recently completed todos first, and if multiple tasks are the same date, they are due by the most recent one.

Supports **bold**, *italic*, [links](url), `code`, etc. 697 chars

Task #2 (1 pt.) – Completed Page URLs

100%

Details

- Paste the direct GitHub file link from the homework branch.
  - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
  - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module04-Homework/public_html/m04/hw/todos/completed.php

URL #2
https://sa3327-it202-450-m2026-prod.onrender.com/m04/hw/todos/completed.php

Section #5: (1 pt.) Misc

67%

Task #1 (0.33 pts.) – GitHub Details

100%

Details

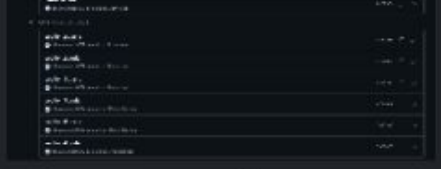
- Show the repository evidence for the Module 04 homework.

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the pull request.
- The screenshot should show at least the baseline commit, planning-comment commits, and solution commits.



commits

Part 2: URLs 100%

Details

- Paste the pull request URL.
  - The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/pull/32/

Task #2 (0.33 pts.) – WakaTime Activity

100%

Details

- Visit the WakaTime dashboard.
- Open `Projects`.
- Find your repository.
- Screenshot the overall time near the repository name (top of page).
- Screenshot the file-level activity (near the bottom).
- The purpose is evidence of activity; the duration isn't tied to the grade.
- The visual graphs are not required.

waka

waka files

Task #3 (0.33 pts.) – Reflection

Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

I learned a lot about SQL queries and how to use PDO statements to insert, update and retrieve data. I also learned how to validate that data and calculate values from it.

I learned a lot about SQL queries and how to use PDO statements to insert, update and retrieve data. I also learned how to validate that data and calculate values from it.

Supports **bold**, *italic*, [links](url), `code`, etc. 171 chars

Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

The easiest part was writing the HTML forms. I've used HTML forms before so a lot of it was just muscle memory.

Supports **bold**, *italic*, [links](url), `code`, etc. 111 chars

Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

THe hardest part was writing the SQL queries and making sure that they had the correct conidtions. THe most difficult part was using DATEDIFF and filtering the results.

Supports **bold**, *italic*, [links](url), `code`, etc. 168 chars

Saving...

End of Assignment - submission has been recorded.